NAMMA TWIN

INTELLIGENT INDOOR SPATIAL RECONSTRUCTION & DIGITAL NAVIGATION

Comprehensive Product, Engineering, Research & Competition Playbook

Team	TWOPOINTERS
Team Lead	Tejaswee Rajput
Team Members	Sohana Pilli · Rahul Sharma
Team ID	TM-C9B0AB28A87
Competition ID	NAMM-65BC0445E14A
Competition	Techfest IIT Bombay · Namma Space 2026-27
Prototype baseline	NAMMA TWIN build 0.2.0
Document version	1.0 · 31 October 2026

A single, scalable engine for capturing, reconstructing, exploring and navigating indoor places.

DOCUMENT STATUS: COMPREHENSIVE WORKING SPECIFICATION

IMPORTANT EVIDENCE NOTICE The bundled 0.2.0 demo environment is procedurally generated synthetic geometry. It is not a scan of a real college. The capture-to-COLMAP workflow is documented but has not been executed on team-collected footage. This document separates implemented prototype behaviour, proposed engineering design and evidence still required.

How to use this document

This project bible is intended to be the shared source of truth for the product, architecture, implementation, testing, competition strategy and team handoff. It is deliberately more detailed than the five-page competition abstract. Use it as an engineering specification and execution handbook; do not submit it in place of the required short abstract unless organisers explicitly request supplementary documentation.

Label	Meaning
IMPLEMENTED	Present in the 0.2.0 source package and described from inspected code or project files.
LOCALLY VERIFIED	A test/check passed in the build environment; this does not imply a full browser or production deployment test.
PROPOSED	Recommended design or next-step feature, not represented as completed.
NOT VERIFIED	Code may exist or be planned, but there is no test evidence yet.
NOT DONE	Required artifact, dataset, deployment, or operation is absent.

READ THIS BEFORE CLAIMING FEATURES A synthetic scene proves interaction mechanics, not photogrammetric reconstruction fidelity. A point cloud is a visual point representation, not a mesh collider or a walkable navigation surface. A sample A* grid is not validated navigation for a real building.

Document map

- PART I - Product, challenge and solution definition
- PART II - Requirements, architecture and end-to-end pipeline
- PART III - Capture SOP, reconstruction and spatial registration
- PART IV - Viewer, POIs, search and pathfinding
- PART V - Backend, data contracts, operations and security
- PART VI - Testing, metrics, risks and deployment
- PART VII - Competition strategy, team execution and judge preparation
- APPENDICES - Repository map, commands, logs, glossary and references

Detailed contents

Sections 1-9	Sections 10-18	Sections 19-27
1. Executive summary	10. Spatial registration	19. Testing and evidence
2. Competition brief	11. Browser viewer	20. Deployment operations
3. Product vision and users	12. POIs, labels and search	21. Failure modes and recovery
4. Scope and requirements	13. Walkability and A*	22. Risk register
5. System architecture	14. Multi-floor and AR	23. Competition strategy
6. Pipeline and state machine	15. Backend API	24. Demo and presentation
7. Smartphone capture SOP	16. Data contracts and assets	25. Judge Q&A
8. Video ingestion and frames	17. Privacy and security	26. Team responsibilities
9. COLMAP reconstruction	18. Repository workflow	27. Immediate task board

Appendices: A. Repository map | B. Quick commands | C. Experiment log | D. Glossary | E. Source register | F. Claim-control checklist

PART I - PRODUCT & CHALLENGE DEFINITION

1. Executive summary

NAMMA TWIN is an indoor spatial-engine concept that converts ordinary indoor captures into an interactive digital representation that people can explore remotely and, in later stages, use for indoor wayfinding. The product aims to replace static floor-plan-only experiences with an intuitive spatial interface: a visitor opens a link, enters a 3D environment, searches for a destination, sees where it is, and follows a walkable route through the building.

The underlying product is not merely a 3D viewer. It is a pipeline plus a set of synchronized spatial representations. The visual representation makes the environment understandable; the geometry representation supports collision and walkability; and the semantic representation identifies meaningful places and objects. These representations must be aligned in one coordinate frame for reliable search and navigation.

ONE-LINE PITCH NAMMA TWIN turns indoor spaces into searchable, explorable digital twins - from smartphone capture to browser-based spatial discovery and obstacle-aware navigation.

1.1 The problem

- Indoor visitors often arrive without knowing the building layout, room locations, entrances, facilities or shortest accessible route.
- Conventional 2D floor plans can be stale, abstract, difficult to interpret and disconnected from the lived appearance of the space.
- Businesses and institutions need a digital way to show a physical place without requiring every visitor to be physically present.
- 3D visualizations alone are insufficient: a photorealistic scene does not automatically know which area is a laboratory, where a door connects, or whether a route is physically walkable.
- Manual 3D modelling and semantic annotation do not scale easily across many buildings and frequent layout changes.

1.2 The proposed solution

- A repeatable smartphone capture SOP that collects overlapping views of a real indoor zone.
- An image-processing and reconstruction pipeline that estimates camera poses and scene geometry from the capture.
- A web viewer that lets remote users move through the reconstructed environment using familiar first-person controls.
- A POI and spatial-search layer that names rooms, facilities and objects and lets users jump to them.
- A separate walkability and obstacle model that allows a route planner to find a valid path around walls and furniture.
- A future mobile AR layer that anchors directions to the live camera view after robust localization is available.

1.3 What NAMMA TWIN is and is not

It is	It is not
A system pipeline: capture, reconstruction, registration, semantics, interaction and validation.	Just a static 3D render, a decorative campus model or a floor-plan image.
A browser-based free-roam experience, with navigation available as a separate capability.	A navigation system that assumes every visible point-cloud location is safe to walk on.
A modular system that can ingest new indoor spaces using a documented SOP.	A claim that all colleges or venues have already been mapped.

It is	It is not
A prototype with a synthetic scene used for demonstrable UI and spatial logic.	A real-college digital twin: no team-collected real-site scan is included in build 0.2.0.

1.4 Why three spatial layers are necessary

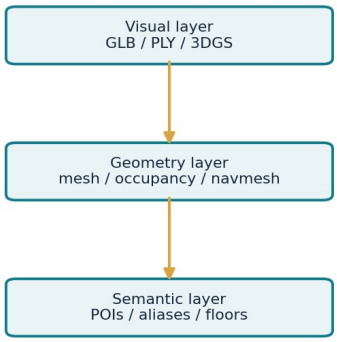


Figure 1. The visual, geometry and semantic layers are distinct but must share one registered coordinate frame.

The visual layer answers “what does this place look like?” The geometry layer answers “where is solid structure and where can a person move?” The semantic layer answers “what is this place and how do people refer to it?” A robust product connects them through a registration transform and stable identifiers. If the layers drift apart, POI markers float away from rooms and routes cross walls.

2. Competition brief and constraints

The official Namma Space problem statement frames an end-to-end intelligent spatial engine: ordinary smartphone images or video of indoor environments become photorealistic, interactive 3D digital twins. Required user capabilities include capture by a non-technical person, reconstruction without manual 3D modelling, free browser exploration, POI selection/tagging, spatial search and routes that remain on the floor and avoid physical obstacles. AR navigation is described as a bonus.

2.1 Round-by-round scope

Round	Official focus	Expected evidence / deliverables	NAMMA TWIN response
Round 1	SOP, 3D reconstruction and free roaming. Dataset: team home or similarly accessible indoor space.	Technical abstract up to 5 pages; working free-roam web-view link; 3-5 minute demo; code repository with setup instructions and sample data in test directory; public-view Drive folder.	Establish capture/reconstruction pipeline and show a working browser walkthrough. Current package demonstrates the viewer using an explicitly synthetic teaching-floor scene; a real capture/reconstruction remains outstanding.
Round 2	POI labels, spatial search and desktop navigation.	Updated repository branch; live app with POI click/search and walkable routes; 3-5 minute demo.	Register POIs to a real scene, build geometry-derived walkability, test obstacle-aware pathfinding.
Round 3	Map a completely new assigned IIT Bombay venue on site and present live.	Day 1: capture, process, reconstruct, annotate and deploy. Day 2: 10-minute pitch/demo and 5-minute technical Q&A.	Build a field-ready pipeline with robust SOP, predictable processing, clear diagnostics and a repeatable deployment path.
Bonus	On-site AR navigation.	Live camera overlay and updating directions.	Only attempt after coordinate registration and device localization are reliable.

2.2 Evaluation criteria to engineer against

Evaluation area	What judges need to see	Evidence to collect
SOP & pipeline soundness	A novice can capture the environment; stages and failure handling are clear.	Capture checklist, sample video, extracted frames, command log, stage timings, error report.
Visual quality & model fidelity	The model preserves room structure, openings, major furniture and spatial continuity.	Input/output examples, coverage review, visual walkthrough, known omissions, measured registration checks.
Viewer performance	Browser loads the scene and supports fluid first-person movement without requiring active navigation.	Device/browser matrix, initial load time, asset size, FPS, memory observations, control test.
Documentation & demo clarity	A reviewer can reproduce the project and understand what is genuinely implemented.	Setup guide, repository, test data, 3-5 minute demo, organized master PDF.

2.3 Competition timeline supplied in the brief

Milestone	Date
Registration deadline	15 October 2026
Round 1 submission deadline	31 October 2026
Round 1 results	10 November 2026
Round 2 submission deadline	30 November 2026
Round 2 finalist announcement	7 December 2026
Round 3 on-site capture and processing	16 December 2026
Round 3 live pitch and technical Q&A	17 December 2026
Prize form deadline	31 December 2026

TIMELINE CONTROL The official brief states these dates. Check the Techfest portal for amendments; the organizer reserves the right to update details.

2.4 Rules and submission hygiene

- The official brief permits open-source libraries and third-party tools, subject to license compliance and credit in the abstract.
- The team must preserve originality: explain which parts are team-built, which use third-party packages and which use external models or sample assets.
- All Drive materials should be readable using “Anyone with the link can view”. Test permissions in a signed-out/private browser.
- Additional URLs and supplementary material must be gathered in one organized master PDF or Google Doc; do not leave loose links in the folder.
- For real captures, obtain location permission and exclude people, personal records, access-control details and restricted spaces from public samples.

3. Product vision, design principles and users

3.1 Product vision and mission

Vision: make every useful indoor place understandable and navigable through a digital spatial interface. Mission: make capture, reconstruction and publishing sufficiently repeatable that a non-specialist can create a digital twin of a permitted indoor zone using a smartphone and a guided workflow.

3.2 Core principles

- Capture first, model second: the quality of reconstruction begins with disciplined image acquisition.
- Separate appearance from geometry: a photorealistic representation is not a collision model.
- Keep semantic anchors stable: POIs need IDs and coordinates that remain consistent through asset updates.
- Fail visibly, never silently: show capture, reconstruction, registration and navigation states and actionable diagnostics.
- Optimize for reproducibility: every demo must be rebuildable from a documented versioned data set and commands.
- Provenance is part of the product: label synthetic, public sample and team-captured environments distinctly.
- Navigation must be conservative: avoid unsafe shortcuts and mark unknown/unmapped regions as unavailable.

3.3 Users and jobs to be done

Persona	Goal	Primary flow	Design implication
Remote visitor	Understand an unfamiliar venue before arrival.	Open link -> explore -> search destination -> inspect route.	Fast load, no install, intuitive controls, clear labels.
On-site visitor	Find a room/facility without decoding a floor plan.	Search -> choose destination -> follow route -> confirm arrival.	Accessible path options, current-location anchor, reroute behaviour.
Facility manager	Publish and maintain an accurate building twin.	Capture -> process -> inspect -> annotate -> publish -> update.	Capture SOP, asset versioning, POI admin, review and audit trail.
Capture operator	Collect usable data without photogrammetry expertise.	Permission -> guided walkthrough -> quality review -> upload.	Simple instructions, coverage feedback and upload recovery.
Developer / integrator	Add new buildings and extend spatial functions.	Ingest asset -> align -> annotate -> build route graph -> test.	Stable contracts, coordinate metadata, versioned APIs and test fixtures.

3.4 Example experience narrative

01 A facility manager selects a floor or room zone and reads the capture checklist.

02 The operator records a slow, overlapping smartphone walkthrough and saves the original file with capture metadata.

03 The pipeline samples frames and produces a report; the operator inspects coverage and blur before reconstruction.

04 COLMAP estimates camera poses and sparse geometry, then dense stereo produces a fused point cloud for inspection.

05 The model is reviewed, scaled and aligned to a building coordinate frame; major floor surfaces and obstacles are prepared for navigation.

06 An editor attaches names and POI coordinates to the registered environment, checks routes, then publishes a versioned viewer link.

07 A visitor opens the link in a browser, explores without navigation, or searches for a destination and sees a route overlay.

PART II - REQUIREMENTS, ARCHITECTURE & PIPELINE

4. Scope and requirements

4.1 Functional requirements

ID	Requirement	Acceptance condition	0.2.0 status
FR-01	Ingest an indoor phone video.	Accept MP4/MOV/M4V/AVI, write to storage, return a job ID and metadata.	Implemented locally; API smoke-tested.
FR-02	Extract useful frames.	Sample frames at configured interval, cap image width, create JPEGs and a machine-readable report.	Implemented; not yet run on team capture.
FR-03	Reconstruct scene geometry.	Run camera pose estimation and dense reconstruction from inspected frames.	COLMAP runner included; real run not executed.
FR-04	Render a 3D scene in browser.	Load bundled GLB sample and optional PLY preview.	Implemented in code; browser test not verified.
FR-05	Free roam without navigation active.	WASD/arrow movement and mouse look work independently of route state.	Implemented in code; browser test not verified.
FR-06	Prevent movement through sample obstacles.	Reject movement when the next position violates sample walkability rectangles.	Implemented for synthetic sample only.
FR-07	Display and search sample POIs.	Filter sample POIs by name, category and description.	Implemented in code.
FR-08	Calculate a sample route.	A* returns grid points for connected sample POIs.	Implemented and backend smoke-tested.
FR-09	Calculate real obstacle-aware routes.	Routes derived from registered real geometry and verified against obstacles.	Not implemented.
FR-10	Deploy a public viewer.	A stable HTTPS URL loads correctly outside local machine.	Not done.
FR-11	Provide mobile AR guidance.	Camera tracking and anchored directional overlays work on site.	Bonus/future; not implemented.

4.2 Non-functional requirements

Quality	Target / principle	How to validate
Usability	A first-time visitor can enter the scene and understand controls with minimal instruction.	Timed first-use test; observe control discovery and errors.
Performance	Viewer remains responsive on declared laptop/browser classes.	Measure initial render, asset transfer, FPS and memory; report hardware.
Reliability	Malformed captures and failed reconstruction stages produce actionable errors.	Negative tests, retry tests, corrupt media and no-feature scenes.
Scalability	Large buildings are partitioned into zones/tiles and streamed by floor or vicinity.	Stress-test representative scene sizes and asset chunking.
Accessibility	Keyboard controls, readable labels, contrast and non-pointer alternatives are supported where possible.	Keyboard-only run and accessibility audit.
Privacy	No unnecessary personal data in captures or public asset files.	Capture checklist, manual review, retention policy and consent.
Maintainability	Stages are modular, documented and versioned.	Clean setup, pinned dependencies, test suite and reproducible outputs.
Safety	No production route is presented as safe until geometry and clearance are validated.	Route verification protocol and explicit unmapped-zone handling.

4.3 Explicit out-of-scope items for the current prototype

- Real-college scan or survey; no such dataset is bundled.
- Automatic conversion from uploaded video to a completed job without an operator invoking COLMAP.
- Metric scale or gravity alignment of the raw COLMAP point cloud.
- A geometry-derived navigation mesh, semantic room segmentation or validated obstacle extraction.
- Live user localization, indoor positioning, multi-floor routing or AR camera overlays.
- Authentication, user/tenant management, durable job queue, production object storage, billing, analytics and production monitoring.

5. System architecture

The architecture is organized as a modular processing pipeline and a spatial application. The capture/reconstruction side prepares assets. A publishing step validates and registers the assets. The application side renders the visual scene and overlays semantic and navigation data. The API exposes ingest, metadata and route operations. For a production deployment, processing should move to a durable asynchronous worker architecture rather than running expensive reconstruction inside a web request.



Figure 2. Logical architecture. The arrows show the intended data flow; production object storage and worker orchestration are proposed, not part of the current local prototype.

5.1 Architecture components

Component	Responsibility	Current technology / status
Capture client	Collect an indoor video, maintain capture metadata and upload.	Smartphone native camera; documented SOP, not a dedicated capture app.
Ingestion API	Validate file type/size, store capture and create an upload record.	FastAPI 0.2.0; local disk; in-memory job map.
Frame preparation	Sample video into overlapping still images and report quality hints.	OpenCV Python script.
Sparse reconstruction	Estimate camera intrinsics/poses and sparse point structure.	COLMAP runner calls feature extraction, matching and mapper.
Dense reconstruction	Estimate dense depth and fuse point cloud.	COLMAP PatchMatch stereo and stereo fusion.
Asset publishing	Validate, normalize, align and store scene asset and spatial metadata.	Manual next step; registration pipeline not completed.
Browser viewer	Render GLB or point-cloud PLY and handle user movement.	Next.js, TypeScript, Three.js; code implemented.
Spatial services	POI index, route requests and walkability queries.	Static sample POIs and A* grid; real-site semantics not ready.
Production data layer	Persist buildings, floors, scans, POIs, route graphs and job state.	Proposed PostgreSQL/PostGIS plus object storage and durable queue.

5.2 Deployment topology - prototype versus target

Concern	Local prototype	Production direction
Frontend	Next.js development server.	Static/SSR deployment on CDN or Next.js host.
Backend	FastAPI on localhost.	Containerized FastAPI behind HTTPS reverse proxy/API gateway.
Media	Local upload directory.	Object storage with signed upload/download URLs.
Jobs	In-process JOBS dictionary.	Persistent job database + queue + dedicated worker processes.
Metadata	Files and Python constants.	PostgreSQL with PostGIS geometry where useful.
Monitoring	Manual terminal output.	Structured logs, traces, error reporting, job metrics and alerts.
Security	Local development CORS.	Auth, authorization, rate limits, file scanning, retention and least privilege.

5.3 Key design decision: decouple the renderer from reconstruction engine

The viewer should consume a published scene package, not call COLMAP directly. This prevents the frontend from depending on a particular reconstruction algorithm. A scene package should include a visual asset, coordinate metadata, semantic layer, navigation layer (when available), asset hashes, source provenance and build/version information. The rendering adapter can support a triangle mesh, a standard point cloud or a dedicated Gaussian-splat format as separate asset types.

6. End-to-end pipeline and state machine



Figure 3. Recommended end-to-end production pipeline from permission through validation.

6.1 Pipeline stage contracts

Stage	Input	Operation	Output	Gate before next stage
0. Authorization	Site and intended scan zone.	Obtain permission; plan path; remove private information.	Capture plan + permission record.	Permission and privacy review complete.
1. Capture	Smartphone.	Perimeter loop, transitions, centre/parallax pass.	Original video + capture metadata.	Full zone covered with overlap.
2. Frame preparation	Video.	Sample at interval, resize, encode and inspect.	Frames + extraction report.	Sharpness/coverage reviewed; unusable spans fixed or documented.
3. Sparse mapping	Frames.	Detect local features, match pairs and estimate camera poses.	Sparse model, camera poses, registered image list.	Sufficient images registered; trajectory and geometry plausible.
4. Dense reconstruction	Sparse model + images.	Undistort images, estimate depth, fuse depth maps.	Dense point cloud / depth maps.	Coverage and obvious artifacts reviewed.

Stage	Input	Operation	Output	Gate before next stage
5. Asset cleanup	Dense output.	Remove outliers, crop, simplify/convert if needed.	Viewer-compatible asset.	File loads, boundaries and visual appearance reviewed.
6. Registration	Asset + reference measurements.	Align axes, scale and origin to a declared frame.	Transform and coordinate metadata.	Scale and floor alignment checked.
7. Navigation prep	Registered geometry.	Extract/author floors, obstacles, portals and clearance.	Occupancy grid or navigation mesh.	Route graph passes collision validation.
8. Semantics	Registered frame + labels.	Attach POI IDs, names, categories and anchors.	Versioned POI index.	Markers overlap intended locations.
9. Publish	All accepted assets.	Bundle, version, host and smoke-test the viewer.	Public scene URL + manifest.	Open from clean browser; no local-only dependencies.
10. Monitor/update	Usage and changed venue.	Review feedback, rescan affected areas and republish.	New scene version + change log.	Old version recoverable; new version validated.

6.2 Job state model for production

State	Meaning	Expected transition
CREATED	A capture job exists but media may not be uploaded.	UPLOADING / FAILED
UPLOADING	Media is being transferred.	UPLOADED / FAILED
UPLOADED	Media stored; no reconstruction started yet.	FRAME_EXTRACTION / CANCELLED
FRAME_EXTRACTION	Frames and quality report are being produced.	FRAME_REVIEW / FAILED
FRAME_REVIEW	Human or automated review gate.	SPARSE_RECON / NEEDS_RECAPTURE
SPARSE_RECON	Camera poses and sparse structure are computed.	DENSE_RECON / FAILED
DENSE_RECON	Dense geometry is being produced.	ASSET_REVIEW / FAILED
ASSET_REVIEW	Asset quality inspection and cleanup.	REGISTRATION / NEEDS_RECAPTURE
REGISTRATION	Coordinate frame, scale and axes are resolved.	SPATIAL_PREP / FAILED
SPATIAL_PREP	Geometry, POIs and walkability are prepared.	READY_FOR_PUBLISH / FAILED
READY_FOR_PUBLISH	All gates pass and version is frozen.	PUBLISHED
PUBLISHED	Scene available through viewer.	SUPERSEDED / ARCHIVED
FAILED / CANCELLED	Terminal state with reason and retry policy.	RETRY as a new attempt or controlled resume.

CURRENT IMPLEMENTATION The current FastAPI `/api/captures` endpoint stops at UPLOADED. It stores a record in process memory and returns a next-step note. It does not run the full state machine or schedule background reconstruction.

PART III - CAPTURE, RECONSTRUCTION & REGISTRATION

7. Smartphone capture SOP

The capture SOP is a core product feature, not merely a field note. The quality of the final model depends on stable camera motion, scene texture, overlap, lighting, coverage and the presence of recognizable visual structure. The SOP should be simple enough for a non-technical operator yet explicit enough to produce repeatable results.

7.1 Pre-capture authorization and preparation

- Obtain permission from the space owner and confirm which rooms, corridors, floors and facilities may be recorded.
- Define one self-contained test zone; do not attempt an entire campus in the first scan.
- Plan a continuous route with the start/end locations, door transitions, corners, narrow areas and zones that need a second pass.
- Choose a period with stable lighting and minimal foot traffic. Ask occupants to leave the capture area when practical.
- Remove or avoid recording faces, badges, student records, whiteboards with personal information, computer screens, access-control panels and security-sensitive infrastructure.
- Clean the lens; use the rear main camera, 1x lens, fixed orientation and stable focus/exposure if the phone supports it.
- Ensure sufficient storage and battery; disable notifications that could appear on screen; do not record audio unless required.
- Record operator, device, lens, resolution, frame rate, duration, lighting and permission in a capture manifest.

7.2 Camera settings and operating guidance

Parameter	Recommended starting point	Reason / caveat
Resolution	Prefer 4K; 1080p is acceptable for a small proof of concept.	Higher resolution may preserve detail but increases storage and processing.
Frame rate	30 fps.	Common baseline; frame extraction later chooses a lower sampling rate.
Lens / zoom	Rear main camera, 1x; do not switch lenses mid-pass.	Avoids discontinuities in focal length and camera calibration.
Height	Approximately 1.0-1.5 m.	Captures room features and typical walking viewpoint; vary slightly only when useful.
Motion	Slow walking; smooth turns; avoid sudden pans.	Reduces motion blur and rolling-shutter distortion.
Overlap	Substantial adjacent-frame overlap; keep corners/door frames visible across frames.	Feature matching needs common visual content.
Lighting	Stable, diffuse lighting; avoid large exposure changes.	Exposure flicker and moving shadows can reduce consistency.
Focus/exposure	Lock if possible; do not refocus continuously.	Reduces frame-to-frame appearance shifts.
Audio	Normally unnecessary; disable to reduce incidental personal data.	The visual pipeline uses images, not speech.

7.3 Capture path - perimeter and centre

01 Start at a distinctive, permitted entrance view. Pause briefly to establish a clear starting image.

02 Walk the perimeter in one continuous loop at a steady pace, facing the walls and room features at an oblique angle rather than repeatedly spinning in place.

03 At each corner, slow down and rotate smoothly so consecutive views overlap across the turn.

04 At each doorway, capture the frame from the first side, pass through slowly, and capture the opposite side with overlapping door-frame context.

05 At corridors and intersections, include enough views before and after the junction to connect the two spaces.

06 Make a second centre pass to add parallax and cover areas hidden from the perimeter path.

07 Include floor-to-ceiling context without excessive rapid tilting. Try to preserve a mostly level camera horizon.

08 Finish with a view overlapping the starting region; this helps the pipeline connect the loop.

7.4 Capture quality checklist

Check	Pass condition	Failure response
Coverage	Every wall/door transition and intended zone appears in multiple views.	Recapture missing sections or mark them intentionally excluded.
Overlap	Distinctive features recur in neighbouring frames.	Slow down and add bridging views.
Sharpness	Frames are not dominated by motion blur.	Reduce walking speed; stabilize hands; inspect frame samples.
Texture	Walls/floors contain identifiable texture or stable details.	Capture oblique views and contextual objects; textureless areas may remain difficult.
Lighting	No major exposure flicker or large motion shadows.	Choose more stable time; avoid moving between bright/dark regions too quickly.
Occlusion	People and moving objects do not block most of the scene.	Wait for clear intervals and repeat the affected pass.
Privacy	No private records, screens, faces or prohibited areas in publishable content.	Exclude/blur/redact or do not publish the capture.
Loop closure	End view overlaps the start region.	Repeat a short closing path before stopping.

7.5 Required capture manifest

```
{
  "capture_id": "site-zone-YYYYMMDD-001",
  "site_label": "permitted internal label",
  "zone_label": "single room or connected zone",
  "operator": "team member / role",
  "permission_status": "approved",
  "device_model": "record exact model",
  "camera_lens": "rear-main-1x",
  "resolution": "3840x2160",
  "fps": 30,
  "duration_seconds": 0,
  "lighting": "natural | artificial | mixed",
  "path_plan": ["perimeter", "doorways", "centre pass", "loop closure"],
  "privacy_review": "passed | needs_redaction",
  "notes": "known omissions or capture issues"
}
```

This is a recommended manifest schema for reproducibility. The 0.2.0 project documentation includes a capture-log checklist, but the manifest is not currently auto-generated by the smartphone or upload client.

8. Video ingestion and frame preparation

8.1 Ingestion behaviour in build 0.2.0

The FastAPI upload endpoint accepts one multipart file under the `file` field. It checks the extension against MP4, MOV, M4V and AVI, creates a UUID, writes the stream to `backend/storage/uploads/` in 1 MiB chunks and enforces a configurable maximum size. `MAX_UPLOAD_MB` defaults to 1500 MB. The endpoint returns a record containing the ID, original filename, size, status `uploaded`, timestamp, stored path, next-step instruction and explicit note that upload is not automatic reconstruction.

8.2 Frame sampling configuration

Parameter	Current default	Meaning
Sampling rate	2 frames per second	Selects one frame at each configured interval; user may tune for motion and scene complexity.
Maximum width	1600 pixels	Large frames are downscaled while preserving aspect ratio.
JPEG quality	94	High-quality JPEG output for frame images.
Quality indicator	Laplacian variance	A blur/sharpness hint; not a formal quality score or acceptance threshold.
Report	`extraction_report.json`	Records source FPS, frame count, duration, sample interval, number of extracted frames and output dimensions.

8.3 Why frame selection matters

Video may contain many near-duplicate frames. Passing every frame into a feature matcher can increase processing time without adding useful parallax. Sampling too sparsely can remove the views needed to connect rooms or turns. The default 2 fps is a starting point, not a universal optimum. A short clip with slow movement and many visual transitions may need a different rate from a long corridor walk.

8.4 Operator review after extraction

- Open the extracted images as a contact sheet and verify the route is visually covered.
- Check corners and doorways for at least several useful overlapping views.
- Look for blur, dark images, overexposure, repeated frames, abrupt orientation changes and privacy-sensitive content.
- Check whether important areas are textureless, reflective, transparent or dominated by moving objects.
- Remove or redact private images before they enter a public test directory; retain original data securely under the agreed retention policy.
- Only run reconstruction after the frame set has passed manual review; preserve the frame list and report for traceability.

LIMITATION Laplacian variance is sensitive to image content and resolution. It can highlight some blurred frames but does not understand whether a frame contains useful geometry, strong overlap, or a good viewpoint. Human review remains necessary.

9. COLMAP reconstruction pipeline

The current baseline uses COLMAP, a Structure-from-Motion (SfM) and Multi-View Stereo (MVS) toolchain. SfM estimates camera poses and sparse scene structure by detecting and matching features across overlapping images. MVS uses the estimated cameras and images to infer dense depth, which is fused into a dense point cloud. This is a conventional and explainable baseline for a first indoor capture pipeline.

9.1 Current COLMAP command sequence

Step	COLMAP stage	Purpose	Expected artifact
1	feature_extractor	Detect image features and estimate camera parameters.	Database with image/keypoint metadata.
2	exhaustive_matcher	Match feature descriptors across image pairs and geometrically verify matches.	Verified image-pair correspondences.

Step	COLMAP stage	Purpose	Expected artifact
3	mapper	Incrementally register images, estimate camera poses and triangulate sparse points.	Sparse model, typically under `sparse/0`.
4	image_undistorter	Prepare images and camera models for dense reconstruction.	Undistorted images + dense workspace.
5	patch_match_stereo	Estimate depth and normal maps from neighbouring views.	Per-image depth/normal maps.
6	stereo_fusion	Fuse depth maps into a dense point cloud.	Fused PLY point cloud.

9.2 Inputs and outputs

Input is a directory of sampled images plus camera data estimated by COLMAP. The sparse model contains registered camera poses and sparse 3D points. The dense stage creates depth maps and a fused point cloud. A PLY file can be inspected in a point-cloud viewer; its existence alone does not prove complete surfaces, correct scale or collision readiness.

9.3 Current run command

```
python reconstruction/extract_frames.py \
  --input path/to/indoor_walkthrough.mp4 \
  --output data/processed/frames \
  --fps 2 --max-width 1600 --jpeg-quality 94

python reconstruction/run_colmap_pipeline.py \
  --images data/processed/frames \
  --workspace data/processed/colmap \
  --copy-to-viewer

# If CUDA SIFT is unavailable on the host:
python reconstruction/run_colmap_pipeline.py \
  --images data/processed/frames \
  --workspace data/processed/colmap \
  --cpu --copy-to-viewer
```

9.4 Reconstruction failure modes and recovery

Symptom	Likely causes	Recovery
Few/no registered images	Insufficient overlap, blur, textureless surfaces, wrong camera grouping or abrupt turns.	Inspect image sequence, add bridging frames, slow capture, retune sampling and rerun sparse mapping.
Disconnected components	Rooms/corridors lack shared visual features or doorway transition is not captured.	Capture both sides of doorway with overlap; process connected zones and register later.
Sparse geometry distorted	Repeated patterns, moving people, reflections, rolling shutter or unstable exposure.	Reduce moving objects, improve lighting stability, inspect matches and remove poor frames.
Dense output incomplete	Poor sparse camera estimates, low texture, reflective surfaces or insufficient parallax.	Fix sparse stage first; recapture difficult regions; adjust dense settings only after validating poses.
Huge point cloud / slow viewer	Too many points, unnecessary distant structure or oversized source images.	Crop, downsample, partition by room/floor and test a level-of-detail strategy.
Wrong orientation/scale	Monocular scale ambiguity or arbitrary reconstruction coordinate frame.	Use reference measurement, floor plane and explicit rigid/scale transform; persist it in metadata.
No viewer asset copied	Pipeline failed before fusion or output path differs.	Check command log, workspace directory and non-empty fused PLY; never substitute unrelated geometry.

9.5 Reconstruction quality review

- Coverage: Are all intended rooms, corners, doorways and transitions represented?

- Completeness: Are floors, walls, ceilings, large furniture and room boundaries sufficiently visible?
- Continuity: Are adjacent spaces connected or clearly separated where required?
- Artifacts: Are there floating points, smeared surfaces, duplicated geometry, holes or moving-object ghosts?
- Alignment: Is the floor approximately level? Does vertical direction match the declared world-up axis?
- Scale: Does a known measured dimension match the reconstruction after scaling?
- Privacy: Are personal and sensitive items excluded or obscured in public output?
- Performance: Is the asset size appropriate for browser download and target device rendering?

9.6 Optional photorealistic upgrade: 3D Gaussian Splatting

A future quality path is to train a 3D Gaussian Splatting representation from a suitable image sequence and camera poses, for example through a tool such as Nerfstudio Splatfacto. Gaussian splats can provide strong view synthesis and an immersive visual experience. However, this is not implemented in build 0.2.0, and a generic PLY point-cloud loader must not be assumed to render Gaussian-splat PLY correctly. A dedicated compatible splat renderer and export contract would need to be selected and tested.

Even if the visual layer uses Gaussian splats, navigation still needs a separate geometry representation. Splats encode appearance and view-dependent radiance; they are not automatically watertight surfaces, collision shapes or a navmesh. The architecture should never couple route safety to a splat-only scene.

10. Spatial registration and coordinate systems

Coordinate registration is the critical bridge between the visual model, geometry and semantic anchors. COLMAP reconstructions from ordinary monocular images can have an arbitrary scale and an arbitrary orientation/origin. A point cloud that looks upright on screen is not necessarily aligned to gravity, and scaling its largest dimension to a convenient size is only a display normalization—not metric calibration.

10.1 Recommended coordinate convention

Convention	Definition
World up	Y axis is vertical.
Ground plane	X-Z plane; Y is height.
Origin	A reproducible landmark such as a surveyed entrance corner or a floor-control point.
Units	Metres after calibration; before calibration explicitly label as arbitrary reconstruction units.
Heading	Declare a fixed building direction, e.g. main corridor axis or north reference if known.
Transform	Persist source-to-world scale, rotation and translation in a versioned transform record.

10.2 Alignment procedure

01 Choose at least one level floor plane and estimate its normal from geometry or a robust manual reference.

02 Rotate the reconstruction so the floor normal aligns with world +Y; verify wall planes are approximately vertical.

03 Choose one known-length baseline, such as a measured doorway width or corridor segment, to estimate metric scale.

04 Choose a stable origin, ideally a clear building landmark that can be revisited in future scans.

05 Apply a rigid transform plus scale to visual and geometry assets; transform every POI and obstacle coordinate using the same matrix.

06 Persist the transform, reference measurements, operator, method, uncertainty and asset version.

07 Validate at least two additional known distances and check that POIs and route lines visually align with the environment.

10.3 Registration data example

```
{
 "coordinate_system": "building-local-y-up-v1",
 "units": "metres",
 "origin_description": "inside face of main entrance threshold, left corner",
 "source_frame": "colmap-reconstruction-local",
 "scale_factor": 1.0,
 "rotation_matrix_3x3": [[1,0,0],[0,1,0],[0,0,1]],
 "translation_xyz": [0,0,0],
 "reference_measurements": [],
 "registration_status": "UNREGISTERED",
 "uncertainty_note": "Populate only from measured control data"
}
```

CURRENT PROTOTYPE BEHAVIOUR When a team PLY is loaded, the viewer centers and normalizes its maximum dimension for display. That is convenience scaling only; the code does not certify real metric scale, gravity alignment or a building-local coordinate frame.

PART IV - VIEWER, SEMANTICS & INDOOR NAVIGATION

11. Browser-based virtual walkthrough

The browser viewer is the visible product surface and a mandatory Round 1 capability. Users must be able to explore freely even when no destination has been selected and no route is being calculated. The viewer therefore treats first-person navigation as a base interaction mode. POI and route overlays are additive layers, not the engine that drives movement.

11.1 Current viewer implementation

Area	Implemented behaviour in source	Boundary
Rendering	Three.js scene with perspective camera, WebGL renderer and lighting. Loads GLB sample using GLTFLoader.	Browser execution and target-device performance not yet verified.
Point cloud	Checks for `reconstructed_scene.ply`; uses PLYLoader and enters point-cloud preview mode if available.	Standard point cloud only; not Gaussian splats, mesh collision or navigation data.
Movement	WASD and arrow keys move horizontally. Camera orientation uses yaw/pitch; Shift increases movement speed.	No vertical flight, step climbing, slope handling or real building collision.
Mouse look	Click canvas to request pointer lock; mouse motion adjusts view. Esc releases pointer.	Browser permission/security context required; must test deployed HTTPS site.
Reset	R key returns the camera to initial sample view.	Initial transform is hard-coded for sample scene.
Collision	Basic sample collision check rejects movement through defined walkability rectangles.	Sample obstacles are manually authored, not computed from a reconstructed mesh.
POIs	Sample markers/labels and a side panel for selection/search.	Markers are hidden for unregistered real PLY preview.
Route preview	A* path drawn as an overlay for sample grid.	Does not certify route validity in real-world geometry.

11.2 Interaction controls

Input	Action	Usability note
Click inside 3D view	Enter pointer-lock free-roam mode.	Display a visible instruction before first interaction.
W / Up arrow	Move forward relative to view.	Movement should be frame-rate independent.
S / Down arrow	Move backward.	Avoid abrupt teleporting across thin obstacles.
A / Left arrow	Strafe left.	Maintain collision checks for the next avatar position.
D / Right arrow	Strafe right.	Maintain collision checks for the next avatar position.
Mouse movement	Look around using yaw and pitch.	Clamp pitch to avoid camera inversion.
Shift	Increase movement speed.	Use short bursts; do not cause tunnelling through obstacles.
Esc	Release pointer lock.	Restore normal cursor and focus to the UI.
R	Reset to the sample start position.	Reset yaw, pitch and movement state consistently.

11.3 Viewer implementation flow

01 On mount, create scene, background/fog, perspective camera and WebGL renderer.

02 Load the selected visual asset. For the sample, load the GLB. If the real PLY filename exists, use point-cloud preview instead.

03 Configure the camera orientation and starting position.

04 Create a render loop, update movement from keyboard state and apply delta-time scaling.

05 On pointer-lock change, update the UI state and show/hide the control hint.

06 For each proposed movement step, test whether the next X/Z position is walkable in the current sample representation.

07 Update POI markers, selected destination and route overlay when the user changes the selection.

08 Dispose geometry, materials, textures, event listeners and renderer when the component unmounts.

11.4 Required viewer improvements before a judged demonstration

Priority	Improvement	Why it matters	Acceptance test
P0	Complete Next.js install, typecheck and production build.	Source syntax transpilation is not a production build.	`npm run typecheck` and `npm run build` both exit 0 on a clean machine.
P0	Browser test pointer-lock and movement on target browser.	The central Round 1 user experience.	WASD and mouse work in deployed HTTPS app; Esc returns focus; R resets.
P0	Make the start overlay and controls obvious.	First-use friction can undermine evaluation.	A new tester can begin and stop roaming without coaching.
P1	Improve asset loading: progress, error state, retry and file-size notice.	Large 3D assets can take time and fail.	Simulate slow network and bad asset path; state is explicit.
P1	Add graceful low-quality rendering mode.	Laptop/iGPU performance may vary.	Reduced pixel ratio/point count remains usable on lower-tier hardware.
P1	Add scene bounds and robust collision abstraction.	Prevent users leaving the floor or crossing walls.	No movement through sample walls; edges stop safely.
P2	Support scene manifest-driven camera start and coordinate transform.	Avoid hard-coded positions for each new building.	New manifest loads without editing viewer source.
P2	Partition large scenes by floors/rooms and stream.	Full-building assets may exceed browser memory.	Only nearby assets load and transitions remain responsive.

11.5 Performance engineering

- Measure asset download time separately from scene parse time and first interactive frame.
- Record file size, number of GLB primitives/triangles or PLY points, texture dimensions and draw calls.
- Use compressed textures and geometry where compatible; consider Draco/KTX2 only after validating deployment decoder paths.
- For point clouds, crop irrelevant regions and use a point budget or level-of-detail strategy.
- Partition large venues by floor or spatial tiles; do not force every asset into one initial download.
- Use device pixel ratio caps to avoid excessive GPU rendering costs on high-density displays.
- Keep controls frame-rate independent using elapsed time rather than moving a fixed distance per frame.
- Record performance on named hardware and browser, with the scene version and network conditions.

12. Semantic layer: POIs, labels and search

A Point of Interest (POI) is a semantic anchor inside a registered spatial scene. It may describe a room, service, entrance, object, facility or zone. A POI must have a stable identifier, a human-readable name, a category, one or more aliases and a spatial anchor. The anchor is expressed in the building-local coordinate frame and must be transformed with the visual model whenever that model is re-registered.

12.1 Sample POIs in build 0.2.0

ID	Name	Category	Sample coordinates (X,Z)	Interpretation
entrance	Main Entrance	Access	(-7.0, -9.7)	Sample start location.
lecture	Lecture Theatre	Learning	(-8.0, -4.15)	Large teaching area.
lab	Teaching Laboratory	Laboratory	(7.5, -4.15)	Equipment and bench area.
carrels	Study Carrels	Study	(3.0, -7.35)	Individual study zone.
commons	Student Commons	Student Services	(9.7, -6.9)	Informal seating and student services zone.

SAMPLE DATA NOTICE These POI names and coordinates are manually authored for the synthetic teaching-floor model. They are illustrative fixtures, not observed or surveyed college locations.

12.2 Recommended POI schema

```
{
 "poi_id": "lab-001",
 "building_id": "building-demo",
 "floor_id": "floor-0",
 "name": "Teaching Laboratory",
 "aliases": ["lab", "computer lab", "practical room"],
 "category": "Laboratory",
 "description": "Teaching laboratory and equipment benches.",
 "anchor": {"x": 7.5, "y": 0.0, "z": -4.15},
 "anchor_type": "point | polygon-centroid | entrance-anchor",
 "accessibility": {"step_free": null, "restricted": false},
 "visibility": "public",
 "source": "manual_annotation",
 "scene_version": "0.2.0-synthetic",
 "verified_by": null,
 "verified_at": null
}
```

12.3 POI authoring workflow

01 Open the registered scene and choose a clear semantic target.

02 Pick an anchor point at a meaningful location: room centre for room search, door threshold for room entry, object anchor for item search.

03 Assign a stable POI ID that does not depend on mutable display text.

04 Enter the canonical name, aliases, category and concise description.

05 Attach floor/building IDs and any access restrictions.

06 Preview the marker from multiple viewpoints and confirm it is not embedded in a wall or floating above the wrong floor.

07 Run a search test with canonical name, alias, partial phrase and misspelling where relevant.

08 Persist the annotation with author, timestamp, scene version and review status.

12.4 Search behaviour

The current frontend filters sample POIs using a lower-cased concatenation of name, category and description. A production search should support aliases, fuzzy matching, category filters, floor filters, synonyms, access constraints and ranking by relevance. Search should return a stable POI ID, not a free-form coordinate generated from the query.

Search tier	Mechanism	Example	Recommended use
Exact	Canonical name/ID lookup.	“Teaching Laboratory”.	Highest confidence; direct result.
Alias	Curated synonyms and abbreviations.	“Lab”, “computer lab”.	Common language variation.
Prefix/substring	Partial string matching.	“lect” -> Lecture Theatre.	Quick type-ahead.
Fuzzy	Edit-distance or token similarity.	“lectur thtre”.	Typo tolerance; use conservative thresholds.
Semantic	Embedding-based retrieval over descriptions and synonyms.	“place to study quietly”.	Future extension; must rank and disambiguate.
Contextual	Filter by floor, building, current zone and accessibility.	“nearest accessible restroom”.	Future on-site visitor experience.

12.5 Search-to-scene interaction

01 User enters a query; search results list matching POIs with names and categories.

02 User selects a result; viewer focuses or teleports the camera to a safe viewing location near the POI (not into the object).

03 The selected marker and label are highlighted. If navigation is requested, the route engine uses the current location or an explicit start POI.

04 If multiple results are ambiguous, show choices instead of silently choosing one.

05 If a destination is inaccessible, unmapped or on an unconnected floor, explain why no route is available.

13. Walkability, collision and A* pathfinding

Indoor navigation is not a shortest-line problem. A route must remain on a walkable floor surface, avoid obstacles and respect connectivity through doors, corridors and floor transitions. The visual model does not automatically provide those guarantees. The system needs a dedicated geometry-derived or carefully reviewed navigation representation.

13.1 What is implemented today

- The synthetic sample uses a rectangular X/Z navigation domain with bounds approximately X=-14.4..14.4 and Z=-10.1..10.7.
- Grid spacing is 0.5 m in the synthetic scene coordinate convention.
- Obstacles are manually authored axis-aligned rectangles, including sample furniture/partitions.
- A* uses four-connected neighbours (north/south/east/west), unit movement cost and Manhattan heuristic.
- The backend has a 20,000-expansion guard; it returns an empty path if no route is found within the search.
- Route distance is computed from consecutive X/Z grid points and returned in a field named `distance_m` for the synthetic sample.
- The frontend includes basic walkability collision checks with an approximate 0.24-unit avatar clearance around the candidate position.

METRIC CAVEAT The sample model uses metre-like coordinates, but its rectangles and grid are procedurally authored. The label `distance_m` is meaningful only within this sample coordinate convention. Do not generalize this to a calibrated real building until scale and registration are measured.

13.2 A* algorithm detail

A* searches a graph from a start node to a goal node using the evaluation function f(n)=g(n)+h(n). Here g(n) is the known cost from the start to node n, and h(n) is an estimate of remaining cost. With four-neighbour movement and equal edge cost, Manhattan distance is an admissible heuristic. The implementation stores predecessors, reconstructs the path when the goal is reached and returns a sequence of grid-cell coordinates.

```
f(n) = g(n) + h(n)

For a 4-connected grid with cell width s:
 g(n) = number_of_grid_steps_from_start
 h(n) = abs(x_index - goal_x_index)
     + abs(z_index - goal_z_index)
 path_distance = sum(distance between consecutive world points)
```

13.3 Why A* on a hand-authored grid is a prototype, not a real navigation system

- The current obstacles are not inferred from reconstructed geometry.
- The grid does not know whether a visible surface is a floor, wall, table top, staircase or void.
- The current graph is single-floor and has no stairs/elevators/ramps or floor transition links.
- Four-connected movement can produce axis-aligned paths; an 8-connected or polygon navmesh can be smoother but must prevent diagonal corner cutting.
- The current implementation does not encode door width, one-way access, accessibility constraints, temporary closures or dynamic obstacles.
- A point-cloud preview hides sample POIs/routes rather than pretending sample coordinates align to the new scan.

13.4 Recommended production representation options

Representation	How it works	Strengths	Trade-offs
Occupancy grid	Discretize floor area into cells classified walkable/occupied/unknown.	Simple, explainable, easy to run A* and inflate obstacles.	Memory grows with resolution; geometry rasterization and noise handling needed.
Navigation mesh	Polygonal walkable surface plus adjacency/portal graph.	Natural continuous paths; widely used in game navigation.	Needs clean floor/obstacle surfaces and robust mesh generation.
Visibility / waypoint graph	Nodes at doorways, junctions and destinations; edges represent safe corridors.	Compact and interpretable for structured buildings.	Requires carefully curated nodes and edges; not as automatic.
Hybrid	Navmesh for local movement, portal graph for room/floor connectivity.	Scales well to large and multi-floor buildings.	More engineering complexity and consistency checks.

13.5 Recommended geometry-to-navigation pipeline

01 Align and scale the reconstruction; estimate the ground plane and identify vertical direction.

02 Extract or manually validate the floor surface. If the reconstruction is point-cloud-only, derive surfaces using a separate meshing/segmentation step or use floor-plan constraints.

03 Extract obstacles from walls, fixed furniture, columns, counters and other non-walkable geometry. Human review should approve critical boundaries.

04 Project the floor and obstacles into a 2D per-floor representation where suitable; preserve height/level metadata for ramps, stairs and multi-storey links.

05 Inflate obstacles by a clearance margin that accounts for the target user/avatar width plus tolerance.

06 Mark unknown areas as blocked until reviewed; do not treat missing geometry as free space.

07 Build an occupancy grid, navmesh or hybrid graph and connect valid doorway portals.

08 Run route queries and then validate the resulting path against the original 3D geometry and clearance rules.

09 Visualize the route as a floor-following line with clear start, endpoint, direction and uncertainty indicators.

13.6 Navigation validation cases

Test case	Expected result
Open corridor	Find a connected path with no obstacle intersection.

Test case	Expected result
Wall blocks direct path	Route detours around a valid opening rather than crossing the wall.
Doorway too narrow	Reject the route for the selected clearance profile or report inaccessible.
Destination inside obstacle	Return invalid destination; do not route to an interior obstacle cell.
Disconnected room	Return no path with a clear reason.
Unknown/uncertain region	Treat as unavailable until reviewed.
Start near boundary	Keep route inside the walkable floor and preserve avatar clearance.
Two floors	Use explicit stair/elevator/portal transition with known accessibility properties.
Dynamic blockage	Trigger replan only when a verified temporary obstacle state is available.
Reverse route	Check whether the route graph is directed or bidirectional and whether accessibility constraints differ.

13.7 Smoothing and route presentation

A raw grid path can contain many short orthogonal segments. A production route can be simplified using line-of-sight string-pulling or a funnel algorithm over a navmesh, but smoothing must be checked against obstacles after modification. The route should be rendered close to the floor surface, not at arbitrary world height. Add a small vertical offset only to avoid z-fighting. Show distance and estimated time only when calibrated movement assumptions exist; otherwise show route length in registered units and avoid implying a reliable walking ETA.

14. Multi-floor and AR navigation

14.1 Multi-floor extension

Model each floor as its own coordinate layer with floor elevation, floor-to-floor transitions and a common building frame. Stairs, elevators, escalators and ramps become explicit transition edges in a higher-level graph. A route can consist of local floor paths connected through a transition node. Accessibility constraints can exclude stairs or select lifts where available.

14.2 AR navigation is a bonus, not a prerequisite

AR requires current user pose in the same coordinate frame as the digital twin. Camera view alone is not sufficient. A phone must estimate its position and orientation relative to the building model, using visual-inertial tracking, relocalization, visual landmarks, QR/image anchors or other site-specific localization methods. Direction arrows must be anchored to a stable world frame and updated as the user moves.

14.3 AR prerequisites and failure handling

- A registered building frame and a map that contains visually recognizable stable landmarks.
- A camera calibration and device pose estimate with uncertainty.
- A strategy for initial localization and recovery after tracking loss.
- A route graph and a current position estimate accurate enough for the corridor scale.
- An explicit state when tracking is lost: hide misleading arrows, ask the user to relocalize and continue only after confidence recovers.
- On-device privacy controls and a clear statement about whether camera images are stored or transmitted.

PART V - BACKEND, DATA CONTRACTS & OPERATIONS

15. Backend API and service design

The backend currently uses FastAPI and Pydantic. It serves health and sample metadata, handles sample route requests and accepts capture uploads. The service is appropriate for local prototyping, but a production pipeline requires durable job state, authentication, storage controls and background processing.

15.1 Implemented endpoints

Method	Path	Request	Response / behaviour	Status
GET	/health	None.	Service name, status and version.	Implemented; smoke-tested.
GET	/api/scene	None.	Reads synthetic scene manifest from data/test.	Implemented; smoke-tested.
GET	/api/pois	None.	Returns static sample POI objects.	Implemented; smoke-tested.
POST	/api/routes	JSON: start_poi (default entrance), destination_poi.	A* path, start/destination POI, distance_m, sample-grid model marker.	Implemented; smoke-tested.
POST	/api/captures	Multipart `file`; extension allow-list.	UUID job, filename, size, status uploaded, timestamp, stored path and next step.	Implemented; smoke-tested indirectly via code; endpoint upload details should be tested on Windows.
GET	/api/jobs	None.	Returns in-memory job records.	Implemented.
GET	/api/jobs/{job_id}	Path job UUID.	One job record or 404.	Implemented.

15.2 Upload validation and operational boundaries

- Only file extension is checked; production should also inspect MIME/container signature and reject disguised or malformed media.
- Upload is streamed in 1 MiB chunks to avoid reading the entire file into RAM.
- Maximum file size is configurable with `MAX_UPLOAD_MB`, default 1500.
- Temporary partial files are removed when upload fails or exceeds the limit.
- The endpoint writes into the backend local storage directory; it is not a cloud object store.
- Job dictionary is process memory; records disappear after backend restart and are not safe across multiple workers.
- The endpoint does not enqueue COLMAP. A team member must run frame extraction and the COLMAP runner separately.

15.3 Recommended production API surface

Method/path	Purpose	Notes
POST /v1/buildings	Create building/venue record.	Permission, owner and metadata.
POST /v1/buildings/{id}/floors	Create floor record.	Declare elevation and floor frame.
POST /v1/captures/presign	Request upload URL.	Return signed object-storage URL.
POST /v1/captures/complete	Confirm upload and create durable job.	Hash, file size, content type, job state.
GET /v1/jobs/{id}	Inspect processing state.	Stage progress, logs, timestamps and actionable failure.
POST /v1/jobs/{id}/retry	Retry eligible stage.	Idempotent and audit logged.
GET /v1/scenes/{id}/manifest	Get published assets and transforms.	Versioned immutable manifest.
GET /v1/scenes/{id}/pois	Search/list POIs.	Filters, pagination and access control.
POST /v1/scenes/{id}/routes	Compute route.	Start pose/anchor, destination POI and access profile.
POST /v1/scenes/{id}/pois	Create/update POI.	Admin authorization and version audit.

Method/path	Purpose	Notes
POST /v1/scenes/{id}/publish	Publish accepted scene version.	Preflight validation and rollback option.

15.4 Job processing architecture

The scalable approach is to separate API request handling from compute-intensive reconstruction. The API accepts an upload and creates a persistent job row. A queue dispatches frame extraction and reconstruction tasks to worker containers. Workers write artifacts to object storage and update job states. A review/publish service validates the scene and exposes an immutable manifest. Long-running COLMAP or Gaussian training jobs should not be attached to a short HTTP request lifecycle.

15.5 Storage model

Entity / object	Key fields or content	Why it exists
Building	building_id, name, owner, metadata, status.	Top-level space identity.
Floor	floor_id, building_id, level, elevation, frame ID.	Multi-floor topology.
Capture	capture_id, device, file URI, resolution, fps, duration, permission, privacy review.	Input provenance.
Processing job	job_id, state, stage, progress, worker, errors, timestamps.	Durable operation tracking.
Scene version	scene_id, version, asset URIs, hashes, transforms, validation state.	Reproducible published twin.
POI	poi_id, floor_id, name, aliases, category, anchor, metadata.	Semantic lookup.
Navigation layer	floor grid/navmesh, obstacle IDs, clearance profile, connectivity edges.	Walkable path calculation.
Audit event	actor, action, old/new version, timestamp, reason.	Accountability and rollback.

16. Data contracts and asset package

16.1 Recommended scene bundle

```
scene-package/
  manifest.json              # scene identity, version, provenance
  visual/
    scene.glb                # mesh/textured visual representation
    scene_points.ply         # optional standard point cloud
    scene.splat              # optional dedicated splat format
  geometry/
    floor_surfaces.glb       # floor/wall/obstacle geometry
    walkability.json         # grid or navmesh reference
    obstacles.geojson        # optional 2D floor-level obstacle polygons
  semantic/
    pois.json                # POI IDs, labels and anchors
    categories.json          # controlled vocabulary
  registration/
    transform.json           # source-to-building coordinate transform
    control_points.json      # known measurements/anchors
  evidence/
    quality_report.json      # capture + reconstruction metrics
    validation.json          # route and scene QA results
  provenance/
    capture_manifest.json    # source capture details
    licences.json            # third-party assets/software/models
```

16.2 Scene manifest example

```
{
 "scene_id": "demo-teaching-floor",
 "scene_version": "0.2.0-synthetic",
 "display_name": "University Teaching Floor - Demo",
 "provenance": {
  "type": "procedural_synthetic",
  "is_real_site_capture": false,
  "source_capture_id": null
 },
 "coordinate_system": {
  "up_axis": "Y",
  "units": "synthetic_scene_units",
  "registration_status": "sample-authored"
 },
 "assets": {
  "visual_glb": "/models/namma_university_teaching_floor.glb",
  "real_point_cloud": null,
  "navigation_layer": "sample-grid"
 },
 "capabilities": {
  "free_roam": true,
  "poi_search": true,
  "sample_route_preview": true,
  "real_geometry_navigation": false,
  "ar_navigation": false
 }
}
```

16.3 Asset format decisions

Format	Purpose	Use and caveats
MP4/MOV/M4V/AVI	Original capture.	Store as private source; record hash, capture metadata and permission.
JPEG/PNG	Sampled images.	COLMAP input; preserve order/filenames and report sampling config.
COLMAP sparse model	Camera poses and sparse reconstruction.	Critical for diagnosing registration and dense reconstruction.
PLY standard point cloud	Dense point positions and optional colour.	Current optional real viewer preview; no solid surfaces/collision by itself.
GLB/glTF	Browser-friendly mesh/scene hierarchy and materials.	Current synthetic viewer asset; production may use compressed textures/geometry.
Gaussian splat asset	Photorealistic view-dependent representation.	Future only; needs dedicated renderer, compatible export and independent navigation geometry.
JSON	Manifest, POIs, transforms, report and graph metadata.	Versioned schema; validate before publishing.
GeoJSON	Optional 2D polygons/lines for floor-level geometry.	Useful for obstacles/route audit if coordinate reference is explicit.

16.4 Versioning and reproducibility

- Assign a scene version for every published set of visual, geometry and semantic artifacts.
- Hash every uploaded capture and generated asset. Do not mutate an already published scene version in place.
- Record tool versions, command arguments, operating system, CPU/GPU, CUDA and driver details.
- Keep raw capture separate from derived frames and final assets; retain only as long as permission/policy permits.
- Store the exact transform that maps reconstruction coordinates into building-local coordinates.

- When a scene is updated, either reproject old POIs through the new transform or require review; never assume they remain correct.

17. Privacy, security, licensing and responsible deployment

17.1 Privacy and consent

An indoor capture may reveal faces, personal workspaces, student records, computer screens, restricted doors, security equipment or other sensitive details. A technically successful capture can still be unsuitable for public publishing. The capture SOP must include permission, data minimization and redaction, and the publishing gate must include a manual privacy review.

17.2 Security controls for production

Risk	Current exposure	Production mitigation
Unauthorized upload	No full auth/tenant layer in prototype.	Authenticated upload, role-based access and signed URLs.
Malicious media	Extension allow-list only.	MIME/container inspection, antivirus scanning, sandboxed processing and strict resource limits.
Oversized upload	1500 MB configured default; local disk.	Quota, per-user limits, object-store multipart upload and retention.
Path traversal	Random UUID filename limits user filename use.	Continue server-generated object keys; never trust path fragments.
CORS overexposure	Configurable allow-list; defaults localhost only.	Only approved production domains; no wildcard credentials.
Private visual content	No automatic person/screen redaction.	Manual review, optional detection/redaction, owner approval and restricted publication.
Worker compromise/resource exhaustion	No isolated worker runtime.	Container sandbox, CPU/GPU/memory/time quotas, network restrictions.
Stale or tampered scene	No production asset integrity workflow.	Hashes, immutable versioned storage and signed manifest.

17.3 License and attribution

The repository contains a license inventory. Keep it updated for every dependency, sample asset, pretrained model, reconstruction tool and renderer. Verify the license at the exact version used and include attribution where required. Do not reuse a public campus scan, mesh or texture unless the rights allow redistribution and competition demonstration. The procedural sample scene should remain clearly identified as generated sample content.

17.4 Data retention

Define how long raw video, extracted images, sparse/dense models and public viewer assets are retained. Raw captures may contain more private information than the published twin. Provide a deletion process for source footage and derived assets when consent is withdrawn or the site owner requests removal.

18. Repository structure and development workflow

```
NAMMA_TWIN_TWOPOINTERS/
  backend/                FastAPI API, navigation and requirements
  frontend/               Next.js UI, Three.js viewer and models
  reconstruction/         OpenCV frame extraction, COLMAP runner,
                        procedural sample generator
  data/test/              Synthetic sample GLB, floorplan and manifest
  docs/                   Architecture, SOP, runbook, handoff, demo script
```

```
exports/classroom/          Placeholder for team-specific exports
scripts/                    PDF builders, floorplan generator, Windows BATs
tests/                      Smoke test suite
submission/                 Abstract, master links template, status checklist
README.md                   Project overview and run instructions
START_HERE.md               Windows quick-start
TEST_REPORT.md              Verification boundary
```

18.1 Development branches and change discipline

- Keep `main` runnable. Develop reconstruction, viewer and API changes in short feature branches.
- Use descriptive commits that identify behaviour and area, e.g. `feat(viewer): add manifest-driven initial camera`.
- Update README and test report when a dependency, endpoint, asset contract or limitation changes.
- Add a regression test for every route/pathfinding change and every new asset manifest field.
- Keep large raw video out of Git history unless the sample is small, privacy-cleared and deliberately included.
- Before demo, tag the exact commit and scene version used in the recording.

PART VI - TESTING, MEASUREMENT, DEPLOYMENT & RISK

19. Testing and evidence strategy

Testing must be layered. A successful Python test does not prove the frontend renders; a successful browser build does not prove the viewer behaves correctly on a real scan; and a synthetic route test does not prove obstacle avoidance in a real venue. The project must maintain an evidence ledger that records exactly which test ran, on which commit and scene version, on which hardware, and what it demonstrated.

19.1 Current verification report

Check	Result in current environment	Interpretation
Python compilation	Passed for backend, reconstruction and utility scripts.	Syntax validity only.
Synthetic GLB integrity	Passed GLB magic/version/declared-length check.	Archive asset is structurally readable at header level.
Scene provenance	Manifest identifies procedurally generated synthetic geometry.	Not a real college scan.
FastAPI smoke tests	Passed for /health, /api/scene, /api/pois and /api/routes using TestClient.	Local API behaviour for these paths verified.
A* sample routes	Non-empty paths to all five POIs.	Sample grid is connected; not real-site route validation.
TS/TSX syntax transpilation	Passed on four app files.	Does not include full semantic typecheck or package build.
Abstract and master sheet	Four-page abstract and one-page links template rendered.	Draft artifacts; links/evidence still pending.
ZIP integrity	Passed.	Archive can be extracted.
npm install / typecheck / next build	Not verified; dependency retrieval was unavailable in the environment.	Must run on an internet-connected development machine.
Browser interaction / device performance	Not verified.	Manual browser test still required.
COLMAP reconstruction	Not executed.	No team capture is included.
Public deploy / demo video	Not done.	Team action required.

VERIFICATION BOUNDARY These results come from the project TEST_REPORT.md and package review. Do not turn “not verified” into “passed” in a presentation, resume, abstract or judge response.

19.2 Test pyramid

Layer	Examples	Frequency
Unit	A* neighbour generation, obstacle collision, POI schema validation, transform application, file extension checks.	Every code change.
Integration	API upload then job lookup; extraction report schema; COLMAP output discovery; scene manifest loads the intended asset.	Every pipeline change.
End-to-end	Capture -> frames -> COLMAP -> registered asset -> browser load -> free roam -> search -> route.	Every milestone and before competition demo.
Visual QA	Coverage, holes, floor alignment, POI marker overlap, route line staying on floor.	Every scene publication.
Performance + operations	Asset transfer, first frame, FPS, API latency, reconstruction runtime; permission, privacy review, public link and data deletion.	Every target release and before publication.

19.3 Required metrics - never fabricate

Metric	Definition / method	Report context
Capture duration	Elapsed recording time for a defined zone.	Device, fps/resolution, zone dimensions and operator.
Sampled frame count	Number of frames emitted by OpenCV.	Sampling fps, max width, JPEG quality.
Registered image ratio	Registered images / input sample images.	COLMAP version and image set.
Sparse point count	Number of reconstructed sparse 3D points.	Reconstruction workspace and filtering.
Dense point count / PLY size	Number of output points and file size.	Dense settings, crop and cleanup.
Reconstruction wall time	Time by stage: features, matching, mapper, dense stereo, fusion.	Hardware, OS, GPU, driver and COLMAP version.
Metric scale error	Difference between reconstructed and measured reference lengths.	At least two independent reference distances.
POI anchor error	Distance between anchor and surveyed/visually checked target.	Annotation method and reference quality.
Route feasibility	Fraction of test routes that are collision-free and reach destination.	Map version, clearance profile and test set.
Viewer load time	Navigation start to first interactive scene.	Browser, CPU/GPU, network and asset size.
Viewer FPS	Frame rate during predefined movement route.	Named device/browser, resolution and scene complexity.
API latency	p50/p95 route and metadata endpoint response.	Concurrent request count and dataset size.

19.4 Suggested release gates

Gate	Required proof	Decision
G0 - capture accepted	Permission and privacy checks passed; coverage and overlap acceptable.	Proceed or recapture.
G1 - sparse reconstruction accepted	Camera trajectory plausible; sufficient intended frames registered; no disconnected major zone.	Proceed or revise frame set.
G2 - visual asset accepted	Dense output inspected; floor/walls readable; artifact severity documented.	Proceed, crop or recapture.
G3 - registration accepted	Scale, ground plane, origin and transform validated against measurements.	Enable coordinate-dependent features only after pass.
G4 - navigation accepted	Walkability layer reviewed; test route set has no collisions; disconnected areas are marked.	Enable real-site routing or keep routing disabled.
G5 - publish accepted	Viewer loads on clean browser; POIs align; asset/provenance/version visible.	Publish version and archive evidence.

20. Deployment and runtime operations

20.1 Local Windows run procedure

The repository provides Windows batch launchers. The following command sequence is the transparent manual equivalent. Use separate terminals so logs remain visible.

```
# Terminal 1 - backend
cd NAMMA_TWIN_TWOPOINTERS\backend
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# Terminal 2 - frontend
cd NAMMA_TWIN_TWOPOINTERS\frontend
npm install
npm run typecheck
npm run build
```

```
npm run dev

# Open in browser
http://localhost:3000
# API health / interactive docs
http://127.0.0.1:8000/health
http://127.0.0.1:8000/docs
```

20.2 Environment variables

Variable	Component	Purpose	Secret?
MAX_UPLOAD_MB	Backend	Maximum accepted upload size; default 1500.	No.
CORS_ORIGINS	Backend	Comma-separated allow-list of frontend origins.	No.
NEXT_PUBLIC_API_URL	Frontend (recommended deployment config)	Public base URL for browser-to-API requests if/when wired through runtime config. Any NEXT_PUBLIC value is client-visible.	No; never place secrets here.
Storage credentials	Production backend/worker	Access to object storage.	Yes; use server-side secret store.
Database URL	Production backend	Database connection.	Yes; do not publish in frontend env.

The 0.2.0 `.env.example` files are reference configuration. Confirm the actual code reads a variable before assuming that simply adding it changes behaviour. In particular, all `NEXT_PUBLIC_` variables are bundled into client-side code and must not contain secrets.

20.3 Public deployment checklist

- Push the exact tested code to the team-owned repository and record the commit SHA.
- Install dependencies from a clean environment and run semantic TypeScript checks and production build.
- Deploy the frontend and backend separately unless a tested combined hosting design is chosen.
- Configure HTTPS, production CORS origins, upload limits, storage and any environment variables.
- Upload/publish the approved sample scene asset; verify asset paths are case-sensitive and not dependent on local files.
- Open the live URL in a private/incognito browser on a second device/network.
- Test initial load, pointer lock, movement, reset, scene rendering, search, POI marker and sample route.
- Check browser console and backend logs for 404, CORS, WebGL, asset parsing or API errors.
- Confirm the public URL works without login if the competition evaluator must access it; do not share a local localhost URL.
- Capture final deployment screenshots, browser/device details and recording timestamp.

20.4 Demo release freeze

At least one day before recording, freeze the commit and sample assets used in the demo. Avoid introducing untested dependencies or last-minute architectural changes. Keep a known-good fallback commit and local build. If a real PLY is not registered, use the synthetic scene and label it as synthetic; do not overlay sample POIs on an unrelated scan.

21. Failure modes, diagnostics and recovery

Failure	User-visible symptom	Diagnostic data to inspect	Safe response
Upload rejected	HTTP 415/413 or interrupted upload.	File extension, size, endpoint log, MAX_UPLOAD_MB.	Show allowed formats/size; retry with permitted file; do not leave partial file.

Failure	User-visible symptom	Diagnostic data to inspect	Safe response
Frame extraction fails	No frames or report.	Video codec, OpenCV error, input file readability, disk capacity.	Re-encode to compatible format or create a new capture copy.
Insufficient feature matches	Few registered images / no sparse/0.	Frames, feature count, match logs, trajectory.	Improve overlap/texture, retune sample rate, recapture missing transitions.
COLMAP CUDA error	Feature/matching stage exits.	GPU driver, CUDA support, log.	Use `--cpu` if supported and accept slower processing; do not assume GPU acceleration.
Dense output missing	No fused PLY.	Undistorted images, depth maps, stereo fusion log.	Fix sparse/undistort/dense stage; never load a stale PLY as current.
Viewer asset 404	Blank scene or loader error.	Network tab, path, capitalization, public directory.	Correct asset path and redeploy.
WebGL/context issue	Canvas black or browser error.	Console, GPU capabilities, browser.	Try supported browser, reduce pixel ratio and inspect fallback.
POI floats in scene	Marker not attached to room.	Coordinate transform, scene version, floor elevation.	Disable POI overlay until corrected and re-registered.
Route crosses obstacle	Path line intersects wall/furniture.	Grid/navmesh build, clearance, transform, test case.	Disable route, correct walkability and rerun validation.
Lost job state	Job ID returns 404 after restart.	In-memory JOBS reset.	Expected prototype limitation; production requires persistent job store.
Slow load	Long blank period.	Asset size, network, parse time, point count, texture size.	Compress, downsample, partition or add loading progress.
Privacy leak	Identifiable or sensitive details visible.	Source video and rendered asset.	Do not publish; redact/rebuild and re-review.

22. Risk register

Risk	Likelihood / impact	Mitigation and owner
Poor capture quality	Medium / High	SOP, on-site QA and recapture gate. Owner: Sohana.
Disconnected room reconstruction	Medium / High	Capture both sides of transitions; overlap views. Owner: Sohana.
No metric scale	High / High	Measured baseline, floor alignment and persisted transform. Owners: Sohana + Tejaswee.
Visual asset too large	Medium / High	Crop, reduce points, compress and stream by room. Owner: Tejaswee.
False route confidence	High / Critical	Keep sample routing labelled; derive geometry and validate. Owners: Rahul + Tejaswee.
Frontend build failure	Medium / High	Clean install, semantic typecheck and production build. Owner: Tejaswee.
Network deployment failure	Medium / High	Test API/frontend, CORS, asset paths and clean browser. Owners: Rahul + Tejaswee.
Privacy/permission issue	Low-Medium / Critical	Written permission, privacy scan and publish review. Owner: all.
COLMAP resource exhaustion	Medium / Medium-High	Bound zone size, sample frames, monitor disk/GPU, CPU fallback. Owner: Sohana.
Schedule compression	High / High	Prioritize vertical slice, freeze scope and keep fallback. Owner: lead.
Dependency/license incompatibility	Low-Medium / High	Pin versions, inspect licenses, retain attribution. Owner: Rahul.
3DGS renderer incompatibility	Medium / Medium	Keep optional; retain PLY/GLB baseline and test separately. Owners: Tejaswee + Sohana.

PART VII - COMPETITION STRATEGY & TEAM EXECUTION

23. Winning-oriented execution strategy

A strong competition entry is not the one with the longest feature list. It is the one that demonstrates a coherent, reproducible vertical slice, measures its results, identifies failure modes and shows a credible route to the full challenge. The project should optimize for evaluator confidence: every prominent claim should be visible in the app, reproducible from the repository or backed by a measured result.

23.1 Minimum vertical slice

01 Capture one small but connected indoor zone with permission and a high-overlap path.

02 Generate the extraction report and inspect frames; preserve an input manifest.

03 Run COLMAP sparse and dense reconstruction and retain logs and outputs.

04 Review visual fidelity, scale and orientation; publish a real reconstructed asset or clearly distinguish the synthetic demonstration asset.

05 Load the asset in the browser, demonstrate free movement and report load/FPS on named hardware.

06 For Round 2, register POIs and build walkability from a validated geometry layer before showing routing.

07 Package the repo, setup steps, sample data, short abstract, video and master links PDF.

23.2 What to prioritize when time is limited

Priority	Do	Do not
P0	A working free-roam scene with clear controls and a real reproducible build.	Spend time on AR before base viewer and pipeline are stable.
P0	A clean capture SOP and a sample dataset with honest provenance.	Claim that a synthetic model is a real reconstructed site.
P0	A visible, inspectable pipeline with logs and limitations.	Hide failed stages or claim automatic processing when the endpoint only uploads.
P1	Measured visual and performance evidence.	Invent FPS, scale error, runtime or reconstruction accuracy.
P1	POI and route logic for Round 2 after coordinate registration.	Route on unaligned sample coordinates over a real point cloud.
P2	3DGS, multi-floor and AR once baseline is validated.	Add multiple experimental renderers that destabilize the submission.

23.3 Milestone plan from current prototype

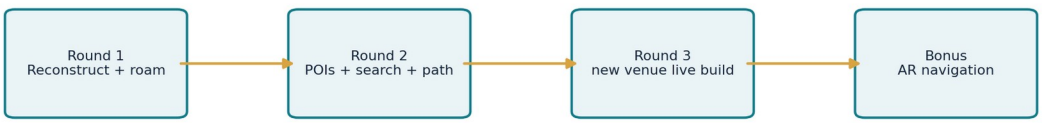


Figure 4. Competition progression. AR is a bonus extension and depends on robust mapping and localization.

Milestone	Deliverable	Definition of done
M0 - build reproducibility	Clean local environment with working backend and frontend.	Clean install, semantic typecheck, production build and documented start commands pass.
M1 - real capture	Privacy-cleared indoor walkthrough and capture manifest.	Coverage reviewed; source capture and sample data permitted for use.
M2 - reconstruction	COLMAP workspace, sparse model, dense PLY and stage log.	Model visually inspected; failure cases and runtime recorded.
M3 - viewer proof	Actual scene asset loaded and free-roam demonstrated.	Clean browser test passes; controls and asset version shown.
M4 - Round 1 package	Abstract, viewer link, video, repo and Drive master PDF.	All links open externally and evidence is internally consistent.
M5 - Round 2 spatial semantics	POI editor/index, search and real registered route layer.	POIs align and route test set passes collision checks.
M6 - Round 3 field run	New venue capture and deployment runbook.	A timed dry run completes from capture to live demo with fallback.

24. Demo and presentation plan

24.1 Round 1 demonstration video - target 3 to 5 minutes

Time	What to show	Narration focus
0:00-0:25	Problem and product identity.	Why indoor digital spaces need exploration, not just flat maps.
0:25-0:55	Capture SOP visual / sequence.	Show the intended smartphone path and explain overlap, doorway transitions and privacy.
0:55-1:35	Pipeline and code evidence.	Show frame extraction report and COLMAP stages. If no real capture exists, explain that this portion is a documented pipeline, not an executed team scan.
1:35-3:20	Browser viewer.	Load the scene, click into the canvas, move using WASD, rotate with mouse, release with Esc and show controls.
3:20-4:10	Sample POIs/search/route if shown.	Explicitly label synthetic sample semantics and route graph.
4:10-4:40	Architecture, limitations and next milestone.	Explain separate visual/geometry/semantic layers and the plan to register a real capture.

Avoid a video that is mostly slides. The judges need to see the actual viewer and interactively explore the scene. Record the screen at a stable resolution, ensure the cursor and keyboard inputs are visible where needed, and include the product URL as on-screen text. The narration must not imply that the current synthetic scene came from a smartphone scan.

24.2 Round 2 demonstration

- Start with a calibrated, registered real scene and its source/scene version label.
- Search for a room using both canonical name and an alias.
- Click a POI label and show its anchor in the scene.
- Select a destination and display a route that bends around a clearly visible obstacle.
- Show at least one route validation case: disconnected destination, blocked opening or alternate accessible route.
- Explain the coordinate frame, walkability source and clearance assumptions.

24.3 Round 3 live demo sequence

01 Show the assigned venue and explain the team's capture plan and permissions.

02 Record capture metadata and demonstrate coverage checks with a rapid sample/contact sheet.

03 Run frame extraction and show reconstruction stage progress and logs.

04 Inspect the reconstructed model and verify scene orientation and scale reference.

05 Register POIs, publish the navigation representation and test a real route around an obstacle.

06 Open the deployed viewer on a second device or browser; demonstrate roam, search and route.

07 State measured processing time, device hardware, model size, viewer performance and known omissions.

25. Judge Q&A preparation

Question	Suggested technically precise answer
How do ordinary phone images become 3D?	We sample overlapping video frames, estimate camera poses and sparse geometry through SfM, then use multi-view stereo to estimate dense depth and fuse a point cloud. The current package includes the extraction and COLMAP orchestration tools; it has not yet been run on a team capture.
Why not use a single AI model end-to-end?	The pipeline separates camera geometry, dense reconstruction, rendering, semantic understanding and navigation so each stage can be validated and replaced. A monolithic model would make failure diagnosis and evidence harder.
Is the current scene from a real college?	No. The bundled teaching-floor environment is procedurally generated synthetic geometry used to demonstrate the browser viewer and sample interaction. It is not a survey or scan of a real campus.
What does the current viewer render?	It loads the synthetic GLB via Three.js. It can also preview a standard point-cloud PLY if the expected file is placed in the public models folder. A Gaussian-splat renderer is not included.
How do you handle arbitrary scale?	The raw monocular reconstruction can have arbitrary scale. We plan to align it using a measured reference length and a floor-plane/axis reference, store the transform, then apply it consistently to assets, POIs and navigation data.
How do you know a route avoids a wall?	In the synthetic prototype, the A* grid uses manually authored obstacle rectangles. For a real building, the walkable layer must be derived or carefully validated from registered geometry, include user clearance and be tested against the original 3D geometry.
Why not route directly on PLY?	A point cloud is a set of samples, not a guaranteed watertight surface or walkable floor. We need a separate floor/obstacle representation such as an occupancy grid or navmesh.
Does the upload endpoint start reconstruction?	No. The current endpoint stores the video and returns a job record. Frame extraction and COLMAP are separate commands; the production design would add a persistent queue and worker.
What is the biggest technical risk?	Reliable reconstruction and coordinate registration in varied indoor spaces, especially textureless or repetitive corridors and doorway transitions. We mitigate this with capture QA, explicit registration and fail-closed navigation.
What will you do when an area is not mapped?	Mark it unknown/unavailable rather than assuming it is walkable. The route engine should not invent a path through missing geometry.
How does this scale to a large building?	Capture and process room/floor zones, register them to one building frame, connect their doorway/vertical transition portals, then stream assets by floor or nearby region.
What about AR?	AR is a bonus stage. It requires device localization in the same coordinate frame as the twin, tracking recovery and uncertainty handling. We will not place arrows until the pose is sufficiently reliable.

26. Team responsibilities and collaboration

The following division is a suggested ownership model based on the team's current technical roles. Confirm the assignment internally; it is a proposed operating plan rather than a record of formally agreed responsibilities.

Member	Suggested primary ownership	Supporting responsibility	Outputs
Tejaswee Rajput - Lead	Product direction, frontend, viewer UX and system integration.	Technical narrative, demo flow, release acceptance and pitch.	Viewer, integration contract, screenshots/video, release checklist.

Member	Suggested primary ownership	Supporting responsibility	Outputs
Sohana Pilli	Capture SOP, frame quality, CV and reconstruction experiments.	Coordinate registration, asset QA and reconstruction measurements.	Capture manifest, image sequence, COLMAP workspace, quality log.
Rahul Sharma	Backend APIs, data model, repository hygiene and documentation.	Deployment configuration, test evidence, master links and handoff.	API, schemas, job tracking, README, reproducible test package.

26.1 Team cadence

- Daily short sync: current blocker, completed artifact, next measurable output.
- Every integration milestone: one member runs the entire clean workflow from documented commands.
- Before a demo: freeze commit/asset versions and maintain a known-good fallback.
- After each capture: upload the capture manifest and extraction report; do not rely on memory or chat notes.
- At each release: update TEST_REPORT.md, submission status and README with any changed limitations.

27. Immediate task board

Order	Task	Owner suggestion	Dependency	Completion evidence
1	Install frontend dependencies on a clean Windows machine; run typecheck and Next.js production build.	Tejaswee	Node/npm registry.	Terminal logs; clean build output.
2	Run the backend with the provided launcher and test all endpoints including actual file upload.	Rahul	Python environment.	API test result and upload record.
3	Open the synthetic viewer in Chrome/Edge and test pointer lock, WASD, collisions, POI selection and route display.	Tejaswee	Frontend build.	Screen recording and manual test checklist.
4	Capture one permitted small indoor space when possible and complete a capture manifest.	Sohana + team	Location permission.	Video + manifest + privacy review.
5	Run frame extraction, review the contact sheet and execute COLMAP.	Sohana	Capture and COLMAP install.	Frames, report, sparse/dense outputs, logs.
6	Register scale, floor orientation and origin; inspect visual asset.	Sohana + Tejaswee	Dense model.	Transform JSON and measured checks.
7	Enable real-scene POIs only after transform; create the first geometry-derived walkability layer.	Rahul + Tejaswee	Registration.	POI file, route graph/grid and route tests.
8	Deploy frontend/backend; test on clean browser and second network.	Rahul + Tejaswee	Build and config.	Public URL, deployment logs, access check.
9	Record 3-5 minute video, finalize abstract and master PDF.	All	Working public demo.	Video, final PDF and public Drive folder.

APPENDICES

Appendix A - Repository map and file responsibilities

Path	Purpose
README.md	Overview, requirements, run commands, API table, limitations and deployment notes.
START_HERE.md	Short Windows quick-start and real capture processing steps.
TEST_REPORT.md	Current verification results and explicitly unverified work.
backend/app/main.py	FastAPI application, CORS, capture upload, manifest, POI and route endpoints.
backend/app/navigation.py	Sample POI constants, obstacle rectangles, walkability and backend A* route search.
backend/requirements.txt	Python backend dependencies.
frontend/app/page.tsx	Main Next.js page and user-facing application shell.
frontend/app/components/SceneViewer.tsx	Three.js viewer, asset load, camera controls, scene modes and overlays.
frontend/app/scene-data.ts	Sample POIs, navigation bounds, obstacle rectangles and frontend A* implementation.
frontend/public/models/namma_university_teaching_floor.glb	Bundled synthetic visual scene.
data/test/namma_university_teaching_floor.glb	Test copy of synthetic GLB asset.
data/test/synthetic_floorplan.png	Illustrative plan image; not a measured building plan.
data/test/synthetic_scene_manifest.json	Scene metadata and synthetic provenance.
reconstruction/extract_frames.py	OpenCV video-to-frame sampling and report generation.
reconstruction/run_colmap_pipeline.py	COLMAP sparse/dense orchestration and optional PLY copy.
reconstruction/build_demo_campus.py	Procedural synthetic GLB generator.
docs/CLASSROOM_CAPTURE_SOP.md	Smartphone capture protocol.
docs/RECONSTRUCTION_RUNBOOK.md	COLMAP install, run, inspect and register procedure.
docs/ARCHITECTURE.md	System architecture notes.
docs/DEMO_VIDEO_SCRIPT.md	Demo flow and narration outline.
submission/	Draft abstract, master links template and deliverable status.

Appendix B - Quick command reference

B.1 Run backend

```
cd backend
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

B.2 Run frontend

```
cd frontend
npm install
npm run typecheck
npm run build
npm run dev
```

B.3 Extract frames and reconstruct

```
python reconstruction/extract_frames.py --input capture.mp4 --output data/processed/frames --fps 2 --max-width 1600
python reconstruction/run_colmap_pipeline.py --images data/processed/frames --workspace data/processed/colmap --copy-to-viewer
```

B.4 Regenerate synthetic model

```
pip install trimesh numpy
python reconstruction/build_demo_campus.py
```

B.5 Smoke tests

```
python tests/smoke_test.py
```

Appendix C - Reconstruction experiment log template

Field	Record for every experiment
Experiment ID / date	Unique run ID and date/time.
Source capture	Capture ID, source file hash, duration, resolution, frame rate.
Capture operator / permission	Operator, site approval and privacy status.
Host environment	OS, CPU, RAM, GPU, VRAM, driver, CUDA, Python.
Tool versions	OpenCV, COLMAP, Node, Three.js, relevant renderer versions.
Frame extraction config	Sampling fps, max width, JPEG quality, output count.
Sparse stage	Input frames, registered frames, sparse points, time and warnings.
Dense stage	Depth maps, fused points, PLY file size, time and errors.
Registration	Scale baseline, floor plane, origin, transform and validation distances.
Viewer	Asset size, load time, browser, device, FPS and interaction test.
Spatial layer	POI count, nav representation, cell size / navmesh version, clearance.
Route tests	Start/end IDs, pass/fail, distance, collision review, notes.
Conclusion	Accepted / needs cleanup / recapture; next action.

Appendix D - Glossary

Term	Meaning in NAMMA TWIN
Digital twin	A digital representation of a physical space with geometry, appearance and contextual data; scope depends on accuracy and update process.
SOP	Standard Operating Procedure; repeatable instructions for capture or operation.
POI	Point of Interest; named room, facility, object or zone with a spatial anchor.
SfM	Structure from Motion; estimates camera poses and sparse 3D structure from overlapping images.
MVS	Multi-View Stereo; estimates dense geometry/depth using multiple calibrated views.
COLMAP	Open-source SfM/MVS pipeline used for camera mapping and dense reconstruction.
Point cloud	A set of 3D points, optionally with colour; not a watertight mesh by default.
Mesh	Vertices, edges and faces representing surfaces.
3D Gaussian Splatting	View-dependent scene representation using many 3D Gaussian primitives for image synthesis.
Coordinate frame	A defined origin, axes, handedness and units used to express spatial positions.
Registration	Alignment of one model or coordinate system to another.
Occupancy grid	A discretized map whose cells are walkable, occupied or unknown.
Navmesh	A polygonal representation of permitted walkable surfaces and their adjacency.
A*	Graph search algorithm using accumulated cost plus heuristic estimate.
Obstacle inflation	Expanding obstacles by user clearance so the path accounts for body width and tolerance.

Term	Meaning in NAMMA TWIN
Pointer lock	Browser API that captures mouse movement for first-person camera control.
GLB	Binary glTF asset format for 3D scenes, geometry and materials.
PLY	Polygon File Format; can store point clouds or mesh data; schema must match the consumer.

Appendix E - Source register and provenance

The project bible combines requirements from the official competition brief with implementation facts inspected from the NAMMA TWIN 0.2.0 archive and clearly marked engineering recommendations. Reference labels below are used throughout this document; they are not tool citation tokens.

Ref	Source	Use in this document
[O1]	Techfest IIT Bombay, Namma Space 2026-27 problem statement, six-page PDF supplied in this conversation (dated 2026-27).	Official challenge objective, mandatory scope, round deliverables, evaluation criteria, rules and schedule.
[P1]	TWOPOINTERS, NAMMA TWIN Round 1 Submission Package, build 0.2.0, generated 31 October 2026.	Current repository structure, implementation facts, sample scene, endpoints, SOP/runbook and test report.
[R1]	COLMAP official tutorial: https://colmap.github.io/tutorial.html	SfM/MVS pipeline terminology and reconstruction stages.
[R2]	COLMAP documentation: https://colmap.github.io/	COLMAP capabilities and command-line reference; use exact installed version when running.
[R3]	Nerfstudio documentation: https://docs.nerf.studio/	Potential future neural rendering / Gaussian splatting tooling; not part of current implementation.
[R4]	Three.js documentation: https://threejs.org/docs/	Browser rendering and loader API reference.
[R5]	Next.js 14 deployment documentation: https://nextjs.org/docs/14/app/building-your-application/deploying	Production build and deployment guidance.
[R6]	Next.js environment variable documentation: https://nextjs.org/docs/14/app/building-your-application/configuring/environment-variables	Client-visible `NEXT_PUBLIC_` variable behaviour and secret-handling caution.

Source discipline: official requirements are attributed to [O1]. Current implementation claims are derived from [P1] and inspected source files. Engineering suggestions such as database choice, worker queues, registration workflow, navmesh, AR and production controls are recommendations, not competition requirements or existing capabilities. Third-party documentation should be checked against the exact package/tool versions selected by the team.

Appendix F - Final claim-control checklist

Proposed statement	Safe to claim now?	Evidence needed for stronger claim
“We have built an interactive synthetic indoor viewer.”	Yes, with qualification that browser production build and live interaction have not been fully verified in this environment.	Clean build and recorded browser walkthrough.
“Our app reconstructs any college from a phone video automatically.”	No.	Real capture, automated job orchestration, successful repeatable reconstructions across multiple spaces.
“The current sample model is a real college digital twin.”	No.	A permitted team-collected capture and reconstruction from that source.
“We have a COLMAP processing runner.”	Yes; included in the source package.	Run it end-to-end and retain logs/output.
“The API automatically starts COLMAP after upload.”	No.	Durable worker/queue integration and end-to-end test.
“We support free roaming in the viewer.”	Code implements the controls; do not claim tested live performance until browser test.	Browser test/video and build.

Proposed statement	Safe to claim now?	Evidence needed for stronger claim
“We have obstacle-aware navigation.”	Only for an illustrative hand-authored synthetic grid.	Geometry-derived navigation layer, coordinate registration and collision test suite.
“We support Gaussian splatting.”	No.	Dedicated compatible renderer, asset export and end-to-end test.
“We support indoor AR navigation.”	No; bonus/future.	Device localization, tracking recovery, AR overlay and field validation.
“This is first-place quality / guaranteed to win.”	No guarantee is supportable.	Competition outcome is determined by judges; focus on evidence and execution.

Closing statement

NAMMA TWIN should be judged by the coherence of its end-to-end spatial pipeline and the honesty of its evidence. The prototype already provides a structured codebase, synthetic environment, viewer interaction design, sample POI and route mechanics, capture tooling and COLMAP runbook. The essential proof points still to be produced are a real permitted capture, a successful reconstruction, coordinate registration, a geometry-derived walkability layer, verified browser performance, public deployment and a clear demo. Completing those in sequence will move the project from a compelling prototype foundation toward a credible indoor spatial engine.